

GABRIEL POPA

Senior Software Engineer - Vue • Python(Django/FastAPI) • AWS

Cluj, Romania | +40 764309991 | gabrielpdev@outlook.com | <https://www.linkedin.com/in/gabriel-popa-12890b414/>

ABOUT ME

Senior Software Engineer with 11+ years of experience building production web platforms across **Python** backend engineering and **Vue.js** frontend development, with strongest hands-on depth in **Python 2.7–3.12**, **Django 1.x–4.x**, **FastAPI**, **Vue 2–3**, **JavaScript/TypeScript**, **PostgreSQL**, **Redis**, **Celery**, **RabbitMQ**, **Docker**, and **AWS**, combining legacy modernization, API design, issue investigation, bug fixing, migration work, runtime upgrades, performance tuning, and new feature delivery to produce stable systems, cleaner releases, and maintainable products across fast-moving business environments.

SKILLS

- **Frontend:** **Vue 2–3**, JavaScript, TypeScript, Vue Router, Pinia, Vuex, Composition API, Options API, reusable component architecture, Vite, Webpack, HTML5, CSS3, Sass, Tailwind CSS, responsive UI, SPA architecture, form handling
- **Backend:** **Python 2.7–3.12**, **Django 1.x–4.x**, **FastAPI**, Flask, REST API design, authentication, background jobs, validation, service integration, data processing, error handling
- **Database / Data & Messaging:** **PostgreSQL**, MySQL, SQLite, **Redis**, **Celery**, **RabbitMQ**, query optimization, indexing, caching, ETL support, reporting workflows
- **Cloud & DevOps:** **AWS**, Docker, Linux, Nginx, GitHub Actions, Jenkins, CI/CD pipelines, deployment automation, environment configuration, containerized deployments
- **Observability / Quality / Security:** OpenTelemetry basics, Prometheus/Grafana basics, Sentry, structured logging, distributed tracing basics, OWASP security practices, rate limiting, **Pytest**, Jest, Vue Test Utils, Supertest, Cypress, Playwright, unit testing, integration testing, contract testing, end-to-end testing, API testing, error monitoring

PROFESSIONAL EXPERIENCE

Senior Software Engineer

Soft Build | Bucharest, Romania (Jan 2024 – May 2026)

- Led modernization of operational workflows by combining **Python 3.11–3.12**, **FastAPI**, and **Vue 3**, replacing harder-to-extend legacy areas with cleaner service boundaries and reusable frontend modules that improved delivery consistency across teams.
- Owned unstable form-heavy screens that regularly produced edge-case validation bugs, then rebuilt shared interaction patterns with **Vue 3**, the **Composition API**, and **TypeScript**, reducing regression volume and improving maintainability in heavily used flows.
- Investigated recurring API failures that surfaced as partial page crashes, then tightened validation rules, normalized error contracts, and improved response handling in **FastAPI**, which made frontend integration safer under degraded backend conditions.
- Addressed slow dashboard behavior by profiling expensive query paths, reducing unnecessary joins, and applying selective **Redis** caching, which improved reporting responsiveness and cut request latency by **35%** on filter-heavy screens.
- Resolved a difficult concurrency issue where overlapping edits occasionally overwrote recent updates, then introduced clearer conflict handling and safer retry logic so users could recover cleanly without silent data loss.
- Fixed repeated timeout problems in export and synchronization workflows by moving long-running operations into **Celery** workers backed by **Redis**, which reduced failed requests and improved operational reliability by **30%**.
- Strengthened weak observability across request and worker flows by adding structured logging, **Sentry**, and basic **OpenTelemetry** instrumentation, making production issues easier to trace across API, UI, and background execution paths.

- Investigated memory growth in a Python batch process by reviewing object retention and execution flow, then split the workload into safer chunks with cleanup points that stabilized worker performance during larger processing runs.
- Handled a postponed runtime upgrade with staged dependency validation and targeted refactors, moving older Python services onto a more supportable baseline without disrupting active release commitments or business-critical workflows.
- Improved release confidence by expanding automated coverage with **Pytest**, **Jest**, **Vue Test Utils**, and **Supertest**, which made refactoring safer and reduced defects in permissions, validation, and workflow-heavy modules by **28%**.
- Applied practical security hardening around sensitive endpoints through stricter request validation, safer authentication handling, and rate limiting, reducing abuse exposure while preserving expected behavior for legitimate user traffic.
- Worked across backend, frontend, deployment, and support concerns in the same delivery cycles, translating evolving requirements into smaller safe releases while also mentoring peers through debugging, rollout discipline, and production-focused design habits.

Software Engineer

Next Apps | Ghent, Belgium (*Nov2020 – Nov 2023*)

- Joined a mixed-style codebase and delivered product features using **Python 3.8–3.11**, **Django**, **FastAPI**, **Vue 2–3**, and **TypeScript**, helping unify implementation patterns without forcing unrealistic rewrites across active modules.
- Took on fragile frontend areas that were difficult to extend, then migrated heavier **Options API** logic toward more maintainable shared composables where appropriate, reducing duplication and making stateful UI behavior easier to reason about.
- Built reusable forms, tables, and validation patterns in **Vue** and **TypeScript**, which reduced copy-paste implementation by **35%** and made feature delivery faster across several customer-facing workflows.
- Supported business-critical third-party integrations where upstream instability caused user-facing failures, then added retry handling, payload normalization, and cleaner logging so external issues became easier to diagnose and contain.
- Investigated inconsistent dashboard totals by tracing update sequencing and **Redis** invalidation behavior, then corrected cache freshness rules and backend ordering so reporting screens became more trustworthy for both users and support staff.
- Resolved stuck-task and duplicate-processing issues in asynchronous workflows by reviewing retry strategy, timeout handling, and cleanup paths in **Celery** and **Redis**, improving job reliability and reducing operational confusion.
- Improved production visibility where meaningful exceptions were being buried in logs by adding **Sentry** and cleaner error mapping across backend and frontend layers, helping teams identify real failures roughly **40%** faster.
- Helped standardize local and deployment environments with **Docker** and cleaner configuration management, which reduced onboarding friction and made production-like issues easier for developers to reproduce before release.
- Identified security gaps on exposed endpoints during feature growth, then applied stronger request validation, authentication checks, and rate limiting that improved resilience against malformed or abusive traffic without adding unnecessary friction.
- Strengthened test coverage around API and service refactors with more disciplined **Pytest** usage and targeted UI checks, which improved change safety and reduced regression risk in asynchronous user journeys by **30%**.
- Contributed across backend delivery, frontend fixes, debugging, deployments, and code reviews while also helping less experienced teammates understand root-cause analysis, test discipline, and safer rollout practices in a busy product environment.

Software Developer

Datamart | Stockholm, Sweden (*Feb 2016 – Sep 2020*)

- Worked on business applications using **Python 3.5–3.8**, **Django**, **Flask**, and early **Vue** adoption, helping evolve server-rendered administrative workflows into more maintainable full-stack solutions without outpacing what the platform could support.
- Supported internal dashboards and reporting tools where usability issues slowed operations, then improved backend behavior and key UI flows so approval, search, and export tasks became more predictable for daily users.

- Reduced repeated frontend rework by introducing reusable table, filter, and modal-edit patterns, improving screen consistency by **30%** and lowering the risk of small regressions during ongoing changes to related modules.
- Helped modernize interaction-heavy areas by moving selected screens from template-heavy rendering toward component-based **Vue** views, making client-side behavior easier to maintain while keeping rollout risk low.
- Solved a persistent reporting bug caused by timezone mismatches between browser input, API handling, and database storage, then standardized parsing and persistence rules to restore reporting accuracy and reduce related errors by **36%**.
- Improved slow reporting workflows by rewriting inefficient **PostgreSQL** queries, reducing repeated reads, and adding missing indexes, which cut unnecessary load and improved response times by **35%** on high-use screens.
- Built and maintained internal REST-style endpoints where frontend and backend concerns had started to blur, then clarified contracts and responsibilities so the application became easier to extend incrementally.
- Reduced strain from long-running imports and scheduled processing by moving heavier tasks into background execution, which stabilized normal request handling and lowered disruption during high administrative activity by **25%**.
- Improved release safety in a process that relied too heavily on manual steps by supporting **Jenkins**, deployment scripts, and environment validation, lowering packaging mistakes by **30%** and making releases more repeatable.
- Made production defects easier to troubleshoot by improving logging discipline, surfacing clearer failure details, and collaborating closely with QA and support so vague reports turned into smaller, safer fixes with faster turnaround.

Junior Software Developer

Berg Software | Timișoara, Romania (Sep2014 – Mar 2016)

- Contributed to a legacy business application built with **Python 2.7–3.5**, **Django**, JavaScript, and early component-based frontend patterns, helping deliver small but important changes without destabilizing tightly coupled production workflows.
- Maintained server-rendered pages, backend forms, and validation logic in modules where stability mattered more than ambitious rewrites, keeping daily user operations dependable while new requirements were introduced gradually.
- Added lightweight interactive UI pieces to selected pages for inline actions, dynamic field updates, and simple status feedback, improving usability while staying compatible with an older template-driven application structure.
- Fixed a recurring login problem that caused unexpected session drops by aligning cookie settings, timeout behavior, and request handling between browser and backend flows, improving sign-in stability by **25%** for regular users.
- Improved form handling across several administrative screens by tightening backend validation, cleaning inconsistent error messages, and correcting edge-case input behavior so failed submissions dropped and recovery became clearer for users.
- Assisted with defect investigation by reproducing bugs from support notes, logs, and tester feedback, then preparing smaller targeted fixes that senior engineers could review and release with less risk.
- Helped improve performance on slower administrative pages by reducing unnecessary queries and simplifying backend data preparation, making frequently used maintenance views more responsive and cutting support complaints by **20%**.
- Supported senior engineers during dependency and framework update work by checking compatibility issues, fixing deprecated code paths, and verifying affected screens so upgrade-related regressions were easier to catch early.
- Added basic tests, extra logging, and shared helper utilities while supporting QA verification and release checks, building stronger engineering habits around debugging, consistency, and safe change management in an early-career role.

EDUCATION

Bachelor's Degree in Computer Science

University of Bucharest | (2010 – 2014)

- Focused on practical software engineering fundamentals that translated directly into building reliable web systems and data-driven applications.

CERTIFICATIONS

AWS Certified Developer – Associate — 2023

- Demonstrated practical capability in developing and maintaining **AWS**-based applications, with focus on deployment, service integration, monitoring, and secure cloud operations.

PCAP – Certified Associate in Python Programming — 2022

- Demonstrated strong foundation in **Python** programming, including core language features, modular design, debugging, and problem-solving for backend-oriented applications.

Scrum Fundamentals Certified (SFC) — 2021

- Demonstrated working knowledge of agile principles, sprint-based execution, team collaboration, and structured delivery practices within software development environments.